

TIM: Adapter Placement by Task Interference Mapping

Alberto Acedo
Biome Makers Inc.

August 2026

Abstract

Low-rank adapters are inserted into the query and value projections of every attention layer. That is the field’s default, and it comes from an ablation in the original LoRA paper rather than from anything measured about the tasks at hand. We give a method, TIM (Task Interference Mapping), for choosing the placement instead of inheriting it. Before any training, TIM passes a few batches of each task through the model, takes principal angles between the two gradient subspaces per module, and adapts the modules where the tasks overlap least. On Llama-3-8B fine-tuned from code to prose, this retains 36.8% more of the model’s original capability than the default placement with the same number of adapted modules (9 of 10 seeds, Wilcoxon $p = 0.006$), with plain LoRA in both arms. The diagnostic takes minutes and trains nothing. Its ordering places `o_proj` first and `q_proj` second across three model families and three task pairs, while the last two positions do not transfer, which is why we present it as a measurement to run rather than a rule to adopt.

1 The problem: a default that does not know your tasks

Fine-tuning a language model on a new task degrades what it already did well. Low-rank adaptation [1] limits the damage by training a small number of added parameters instead of all of them, and it has become the standard way to specialise a model.

Where those parameters go is a choice. The original LoRA paper ablates over which projections to adapt, on GPT-3 and at a fixed parameter budget, and reports that query and value together perform best; the practice has been inherited since and the major implementations default to it.

That choice has been revisited, and by several groups: Section 7 surveys them. What none of them use, as far as we can tell, is the quantity that governs how much a model forgets, which is how much the *two specific tasks at hand* overlap. Existing placement criteria score a module by its own importance, by how much its features would move, or by alignment with the pre-trained weights — all properties of the model and the new task, not of the relationship between the old task and the new one.

This paper asks whether selecting on that relationship works, and whether it can be measured cheaply enough to be worth measuring.

2 The theory it rests on

A recent result makes the question answerable. Steele [2] reports that forgetting under LoRA follows

$$\mathcal{F} = \alpha (1 - \cos^2 \theta_{\min}) + \beta, \tag{1}$$

where $\theta_{\min}$ is the minimum principal angle between the gradient subspaces of the two tasks, validated on Split-CIFAR100 with ViT-LoRA and on sequential GLUE with RoBERTa-LoRA. The relationship is descriptive: how much a model forgets is governed by how much the two tasks share.

Two things follow that the original work does not pursue. The governing quantity is a property of the *pair of tasks and the module*, so it varies across modules within one model. And it can be computed *before* training, because gradients require only a forward and a backward pass and no parameter updates.

Together those give a procedure: measure the overlap per module, adapt where it is smallest. Whether that procedure works is a separate question from whether Equation (1) holds, and it is the one we answer.

3 The method

TIM takes a model and two tasks and returns a per-module map of how much they interfere. Given those, we accumulate the gradient of every candidate weight matrix over n batches of each and take the r -dimensional row space of each gradient by singular value decomposition, with r the LoRA rank that would be used. Two choices in how that comparison is made are not cosmetic, and we state them because the measurement does not work without them.

The statistic is the dimension of overlap, not the leading cosine. The leading principal cosine corresponds to $\theta_{\min}$ and is what Equation (1) uses, but it saturates at one as soon as two subspaces share a single direction: it cannot distinguish sharing one direction from sharing eight. We use $\sum_i \cos^2 \theta_i$, the effective dimension of the overlap. On constructed subspaces sharing k of 8 directions, the leading cosine returns 1.0000 for every $k \geq 1$ while this sum returns 1.03, 2.01, 4.01, 6.00 and 8.00 for $k = 1, 2, 4, 6, 8$.

The reference is a same-task ceiling, not a random-subspace null. Two gradients taken from the same model share directions dictated by that model’s structure regardless of which task produced them, so comparing against random subspaces asks a question whose answer is trivially yes. We split the first task in half and measure the overlap between halves: that is the most two gradients can share with no task difference between them. What informs is the drop from that ceiling. The ceiling itself varies substantially across modules, and since a module with a low ceiling cannot drop far, we rank by the drop *relative* to the ceiling.

Scope of the candidate set. We rank the four attention projections. Nothing in the criterion is specific to them, and other placement work reports MLP modules mattering [6, 10], so extending the candidate set is straightforward; we have not measured that case and do not claim anything about it.

The whole measurement is a forward and backward pass over a few batches, with no optimiser and no parameter updates. It takes minutes on one GPU.

4 Does it work?

We fine-tune Llama-3-8B from code to prose and measure retained code capability by HumanEval pass@1 over ten random seeds; the protocol is in Appendix A. The placement TIM suggests is the quartile of largest relative drop: 17 o_proj, 10 q_proj and 4 k_proj modules, and no v_proj at all — already at odds with the default, since v_proj is one of the two the default adapts. The comparison arm uses query and value projections across layers spread through the model, cut to the *same count* of 31 modules, so that what varies between arms is location and not capacity.

Table 1: Ten seeds, same session, same hardware, plain LoRA in both arms with no penalty of any kind. *Absolute* is HumanEval pass@1 after the new task; the retention ratio divides it by the value after the first task. We report both because the ratio is sensitive to variation in its denominator.

Placement	Modules	Retention	Absolute	vs. default
Default (q,v)	31	54.1%	0.151	—
Measured	31	75.6%	0.206	9/10 , sign $p=0.011$, Wilcoxon $p=0.006$

The gain is +36.8% relative in absolute capability. Both arms end the first task at the same capability (0.274 and 0.285), so the retention ratio is not inflated by a ceiling effect; the small difference runs against the measured arm if anything.

The ordering transfers; its tail does not. Running the diagnostic alone — no training — on Llama-3-8B with two further task pairs, and on Mistral-7B and Qwen2.5-7B with code to prose, puts `o_proj` first and `q_proj` second in all five combinations.

Table 2: Relative drop from the same-task ceiling, averaged per module type. Diagnostic only: no training was performed for these rows.

Model and task pair	<code>o_proj</code>	<code>q_proj</code>	<code>k_proj</code>	<code>v_proj</code>
Llama-3-8B, code → prose	0.751	0.683	0.648	0.584
Llama-3-8B, code → math	0.698	0.681	0.646	0.566
Llama-3-8B, prose → math	0.511	0.403	0.213	0.244
Mistral-7B, code → prose	0.717	0.395	0.394	0.336
Qwen2.5-7B, code → prose	0.592	0.445	0.322	0.367

The last two positions swap in two of the five, and the size of the separation varies widely. Across the three model families on code to prose, the gap between the first and second module type runs from 0.068 on Llama to 0.32 on Mistral, a factor of about five; across all five combinations it falls as low as 0.017, on Llama with code to math. The head of the ordering is not an artefact of one model or one task pair; the tail is not stable and the magnitude is not transferable at all. That is why the diagnostic is something to run on a given model and task pair rather than a fixed recommendation to adopt.

5 Does it combine with a weight-space regulariser?

A separate line of our own work penalises a topological property of the weight graph during training and improves retention on the same protocol [3]. The two act at different points — one chooses where trainable parameters go, the other constrains how they end up — so we asked whether they compose, and measured it rather than assuming.

The answer is that we cannot tell. Adding the penalty on top of the TIM placement gives 6 of 10 seeds on the retention ratio and 7 of 10 on absolute capability, with mean differences of +0.047 and +0.027. Neither approaches significance ($p = 0.85$ and $p = 0.28$) and both sit below the run-to-run standard deviation of 0.104 reported in Section 6. The direction is positive on both measures, and that is the whole of what the data support.

One detail of that experiment generalises beyond it. We first ran the comparison at the penalty strength selected for a configuration with twice as many adapted modules, and the result came out *negative*. A sweep

on held-out seeds showed that strength to be ten times too strong for half the adapted modules, and the sign reversed on recalibration. Any penalty strength chosen for one configuration should be re-derived for another, and a negative result obtained without doing so says nothing.

6 What this does not establish

A second training validation. We attempted one on Mistral-7B and could not complete it: that base model’s HumanEval capability is near zero, so retention has no defined denominator. A second validation needs a model with real capability on the retained task and remains open. Steele’s law was validated on other architectures, but it is a law about forgetting, not about this selection procedure, so it does not stand in for that.

How much variation this setting carries. Repeating an identical configuration — same seed, same code, same hardware — gives a standard deviation of 0.104 in retention ratio, with individual repeats as far apart as 0.596 and 0.793. The seed does not identify the run, so a seed-paired test compares runs differing by more than the seed, and any mean difference below roughly 0.066 is not distinguishable at ten seeds. That GPU non-determinism dominates seed variance is documented in vision [12] and reinforcement learning [13]; we have not found it quantified for low-rank fine-tuning of language models, and it bounds every seed-paired comparison in this literature, this paper included.

7 Where this sits

Adapter placement is an active question and several groups have attacked it. Table 3 separates them by what they measure, when they measure it, and what they optimise, because those axes are what distinguish the approaches and because on two of them this work is alone.

Table 3: Methods that decide where or how to adapt. *Signal* is the quantity scored, *timing* is when it is computed, *objective* is what the method is optimised for. Every entry scores a property of the model or of the new task; none scores the *relationship* between the retained task and the new one, which is what Equation (1) identifies as governing forgetting.

Method	Signal	Timing	Objective
LoRA ablation [1]	downstream accuracy	once, offline	new-task accuracy
AdaLoRA [4]	adapter singular values	during training	new-task accuracy
PLoP [6]	normalised feature norm	before training	new-task accuracy
FLoE [8]	Fisher information	before training	new-task accuracy
Aletheia [9]	gradient magnitude	before training	new-task accuracy
DomLoRA [10]	dominant module	before training	new-task accuracy
Amazon Nova [11]	benchmark sweep	once, offline	accuracy / latency
O-LoRA [5]	past-task subspace	during training	retention
OPLoRA [7]	top- k singular dirs. of W_0	during training	retention
Omega-S [3]	weight-graph topology	during training	retention
TIM (this work)	task-pair gradient overlap	before training	retention

Placement methods optimise for the new task, not for what is kept. The upper block chooses where to adapt, and every one scores for downstream accuracy. PLoP [6] identifies module types by how their feature

norms align with the pre-trained representation, arguing that the least aligned contribute most to adaptation. FLoE [8] selects layers by Fisher information, Aletheia by gradient magnitude [9], DomLoRA by reducing to a single dominant module [10]. These answer a different question: given that I want to learn a new task well and cheaply, where should the capacity go? A module that is ideal for learning fast need not be one where learning does least damage, and none of these works reports retention.

The largest published sweep we know of is Amazon’s on their Nova 2.0 Lite model [11], and it is worth reporting because it agrees with us by another route: they find `o_proj` alone never fails outright on any task, typically lands within a few points of the best configuration, and is an attractive default. Our measurement fills 17 of its 31 slots with `o_proj` modules, arriving there from task overlap rather than from a benchmark sweep.

Retention methods act during training, not on placement. The middle block optimises for what is kept, which is our objective, but all of them intervene while training rather than before it. O-LoRA [5] constrains updates to be orthogonal to past-task subspaces. OPLoRA [7] decomposes the frozen weights by SVD and projects updates into the orthogonal complement of the top- k singular subspace, proving that this preserves the top- k singular triples exactly, and introduces a metric ρ_k for how much an update aligns with dominant directions.

OPLoRA in particular is complementary, not competing. It is the closest work to ours in problem and in tooling — both reason about subspaces, both target forgetting — so the distinction is worth stating precisely. OPLoRA protects the dominant singular directions *of the pre-trained weights*, a quantity that does not depend on which tasks are involved; we measure where the *two tasks’ gradients* overlap, which does. And the two act at different points: OPLoRA modifies the update inside modules that have already been chosen, whereas this work chooses which modules to instrument at all. Doing both — select placement with TIM, then project updates within the selected modules — is coherent and untested, and we would be glad to see it tried.

Why we report no head-to-head numbers. We compare accuracy against none of the above, because as the evaluations currently stand the comparison would not mean anything. OPLoRA fine-tunes LLaMA-2 and Qwen2.5 on MetaMathQA and measures forgetting on ARC and SIQA; PLoP and FLoE report downstream accuracy on their own suites; we fine-tune Llama-3-8B from code to prose and measure HumanEval retention. Putting those numbers in one table would be putting different quantities, measured on different models, in one column. A real comparison means re-running the methods under one protocol. We have not done it, and it is the thing we would most like someone to do.

How much orthogonalisation has to offer here. Steele [2] observes that orthogonalisation-based methods add little where natural orthogonality is already high. Our measurement finds exactly that in seven of eight principal directions, which suggests the room for projection methods varies by setting as much as the room for placement does.

8 Conclusion

The default placement for low-rank adapters is a default: selected once, on one model, and carried since. TIM replaces it with a measurement that takes minutes, and in the case we tested doing so retains a third more capability. The ordering that measurement produces is stable at its head across three model families and three task pairs and unstable at its tail, which makes it a diagnostic to run rather than a rule to copy, and that is how we offer it.

Patent

The method described here is covered by USPTO patent application No. 64/121,656, filed 29 July 2026. The released code is under AGPL-3.0 for non-commercial research and academic use, with evaluation inside a commercial organisation granted explicitly; commercial use is licensed separately.

Availability

Code, per-seed data and a reporting template are at

<https://github.com/BiomeMakers/TIM-OmegaS>. **We would like others to run the diagnostic on models we have not tried**, and a report that the ordering comes out differently is as useful to us as one that confirms it.

A Protocol

Llama-3-8B, LoRA rank 8, $\alpha = 16$, dropout 0.1. Task A is Python from CodeSearchNet, task B is OpenWebText. Capability is HumanEval pass@1, measured after task A and again after task B. Ten seeds: 42, 123, 456, 789, 1011, 2022, 3033, 4044, 5055, 6066. The diagnostic uses 32 batches per task at sequence length 256, split into halves of 16 for the same-task ceiling, and subsamples 1024 rows per module. All arms in Table 1 were measured in one session on the same hardware; per-seed values are in `results/placement.json`.

References

- [1] Hu E J, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.
- [2] Steele B (2026). Subspace Geometry Governs Catastrophic Forgetting in Low-Rank Adaptation. *arXiv:2603.02224*.
- [3] Acedo A (2026). Omega-S: A Functional Resilience Index for LLM Fine-Tuning. *arXiv:2608.03887*.
- [4] Zhang Q, Chen M, Bukharin A, He P, Cheng Y, Chen W, Zhao T (2023). AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. *ICLR*.
- [5] Wang X, Chen T, Ge Q, Xia H, Bao R, Zheng R, Zhang Q, Gui T, Huang X (2023). Orthogonal Subspace Learning for Language Model Continual Learning. *EMNLP Findings*.
- [6] Hayou S, Ghosh N, Yu B (2025). PLoP: Precise LoRA Placement for Efficient Finetuning of Large Models. *arXiv:2506.20629*.
- [7] Xiong Y, Xie X (2025). OPLoRA: Orthogonal Projection LoRA Prevents Catastrophic Forgetting during Parameter-Efficient Fine-Tuning. *arXiv:2510.13003*.
- [8] Wang X, Gao L, Wang H, Zhang Y, Zhao J (2025). FLoE: Fisher-Based Layer Selection for Efficient Sparse Adaptation of Low-Rank Experts. *arXiv:2506.00495*.
- [9] Saket A (2026). Aletheia: Gradient-Guided Layer Selection for Efficient LoRA Fine-Tuning Across Architectures. *arXiv:2604.15351*.

- [10] Zhang S, He R, Fang D, Tan X, Chen K, Zhuang H (2026). Rethinking Adapter Placement: A Dominant Adaptation Module Perspective. *arXiv:2605.06183*.
- [11] Amazon Science (2026). Optimizing LoRA target module selection for efficient fine tuning. *Amazon Science Blog*. <https://www.amazon.science/blog/optimizing-lora-target-module-selection-for-efficient-fine-tuning>
- [12] Morin M, Willetts M (2020). Non-determinism in TensorFlow ResNets. *arXiv:2001.11396*.
- [13] Nagarajan P, Warnell G, Stone P (2018). Deterministic Implementations for Reproducibility in Deep Reinforcement Learning. *arXiv:1809.05676*.